2026-11-09 무한 스크롤과 Spring fetch

- 날짜: 2026-11-09
- 앱: `lgcns-fe-next`
- 기준 코드: `app/(post)/posts/page.tsx`, `components/templates/InfinityWrapper.tsx`, `components/organisms/PostInfinityScrollList.tsx`, `components/organisms/useAdjacentPostPage.ts`, `components/organisms/usePreservePrependScroll.ts`, `components/organisms/useVisiblePostPage.ts`, `lib/postPageWindow.ts`, `lib/postsListHref.ts`, `lib/postCache.ts`, `service/postService.ts`의 `listPosts`, `lib/fetchClients.ts`, `config/apiuri.ts`, `service/spring/sprignAuthService.ts`, `service/auth/springAuthAdapter.ts`, `service/auth/prismaAuthAdapter.ts`, `service/auth/authPort.ts`, `actions/authActions.ts`, `lib/authOptions.ts`, `lib/accessJwt.ts`, `app/api/auth/login/route.ts`, `app/api/auth/[...nextauth]/route.ts`, `types/response/springApiResponseType.ts`, `types/response/ResponseType.ts`, `types/next-auth.d.ts`
- 이 문서는 그 파일이 지금 디스크에 있는 동작만 적는다.

글 목록의 무한 스크롤은 Spring을 호출하지 않는다. 한 페이지는 서버의 `getCachedPosts`가 Prisma로 읽고, 브라우저는 그 페이지를 주소의 `page`로 받은 뒤 이미 들고 있는 페이지 묶음에 붙인다. Spring으로 바뀐 호출은 자격 증명 로그인이다. `AUTH_ADAPTER`가 `prisma`가 아니면 `springFetch`가 `NEXT_PUBLIC_API_URL` 아래 `/api/v1/auth/login`으로 POST한다. `config/apiuri.ts`의 글 경로 문자열은 아직 어떤 함수도 호출하지 않는다.

1. 파일이 언제 실행되는가

`app/(post)/posts/page.tsx`와 `app/layout.tsx`에는 이 기능을 위한 `<script>`가 없다. `async`, `defer`, `type="module"` 속성을 단 스크립트 태그도 없다.

파일	선언	실행 시점	async / defer / type="module"과의 차이
app/(post)/posts/page.tsx	export default function PostsPage. 안쪽 async function PostsContent	서버가 /posts 요청을 그릴 때 searchParams와 getCachedPosts를 기다린다. 목록 조회는 <Suspense> 자식이다	서버 컴포넌트다. HTML <script>가 아니다
lib/postCache.ts	export async function getCachedPosts. 본문에 "use cache"	PostsContent가 서버에서 기다릴 때 캐시 엔트리를 찾거나 postService.listPosts를 실행한다	함수의 async는 ECMAScript 비동기 함수다. HTML async 속성이 아니다
service/postService.ts	async function listPosts. 바깥에서는 postService.listPosts로 호출된다	캐시가 미스일 때 서버 프로세스에서 Prisma count와 findMany를 실행한다	브라우저 모듈 그래프에 들어가지 않는다
components/templates/InfinityWrapper.tsx	export default function PostInfinityWrapper. "use client" 없음	서버 컴포넌트로 props만 넘긴다	서버 컴포넌트다
components/organisms/PostInfinityScrollList.tsx	파일 첫 줄 "use client". export default function PostInfinityScrollList	서버 HTML이 나온 뒤 브라우저가 클라이언트 번들을 받아 hydrate할 때 함수가 다시 실행된다	"use client"는 <script type="module">이 아니다. 이 파일 경로를 src로 받는 태그는 없다
components/organisms/useAdjacentPostPage.ts	"use client". export function useAdjacentPostPage. useEffect 안에서 new IntersectionObserver	hydrate가 커밋된 뒤 effect가 관찰을 시작한다. 언마운트와 의존값이 바뀌면 disconnect한다	IntersectionObserver는 DOM API다. 스크립트 로드 속성으로 붙지 않는다
components/organisms/usePreservePrependScroll.ts	"use client". useLayoutEffect	이전 페이지를 앞에 붙인 뒤, 브라우저가 그리기 전에 scrollBy로 높이를 맞춘다	useLayoutEffect는 페인트 전 동기 레이아웃 단계다. defer 스크립트가 아니다
components/organisms/useVisiblePostPage.ts	"use client". useEffect로 scroll 리스너	hydrate 뒤 스크롤할 때마다 data-post-page 요소의 위치를 읽는다	리스너는 { passive: true }다. 스크롤을 막지 않는다
lib/postPageWindow.ts	export function createPostPageWindow, reducePostPageWindow. async 아님	클라이언트 렌더 중에 동기 함수로 페이지 묶음을 계산한다	네트워크 함수가 아니다
lib/fetchClients.ts	export function createFetchClient. export const springFetch. 안쪽 async function request	모듈이 평가될 때 springFetch 객체 하나가 만들어진다. request는 springAuthLogin이나 springUserGetData가 await할 때 실행된다	fetch는 Fetch 표준 API다. <script>가 아니다
service/spring/sprignAuthService.ts	export async function springAuthLogin, springUserGetData	서버에서 어댑터나, 나중에 이 함수를 부른 코드가 기다릴 때 실행된다. 파일 이름의 sprign은 디스크에 있는 철자다	서버 모듈이다
actions/authActions.ts	파일 첫 줄 "use server". export async function loginWithCredentials	NextAuth authorize와 POST /api/auth/login이 호출할 때 서버에서 실행된다	"use server"는 번들러 지시문이다. HTML async가 아니다

파일	선언	실행 시점	async / defer / type="module"과의 차이
lib/authOptions.ts	export const authOptions. async authorize, async jwt, async session	app/api/auth/[...nextauth]/route.ts의 GET/POST가 NextAuth에 이 객체를 넘긴 뒤, 로그인 요청이 콜백을 부른다	Route Handler다
app/api/auth/login/route.ts	export function OPTIONS. export async function POST	HTML/Vite가 POST /api/auth/login을 보낼 때 서버에서 실행된다	응답은 JSON이다. defer 스크립트가 아니다

HTML에서 async 스크립트는 받는 즉시 실행하고 문서 파싱을 막지 않으며 순서가 없다. defer는 파싱을 막지 않고, 문서 파싱이 끝난 뒤 적힌 순서대로 실행한다. type="module"은 그 지연 실행에 import/export가 더해진다. 이 두 기능은 그 세 속성으로 붙지 않는다. 목록의 첫 페이지는 서버 요청 안에서 만들어지고, 다음 페이지 관찰은 hydrate 이후 effect에서 시작된다. Spring 로그인의 fetch도 브라우저 스크립트가 아니라 서버 함수 안에서 나간다.

참고: HTML script 요소, MDN script, ECMA-262 비동기 함수, Next.js use client, Next.js use cache.

2. 무한 스크롤이 페이지를 받는 방법

/posts의 검색 파라미터는 page, sort, q다. PostsContent는 page가 1 이상의 정수가 아니면 1로 읽고, sort가 문자열 ASC일 때만 ASC, 아니면 DESC로 읽는다. q는 normalizePostKeyword로 앞뒤 공백을 지우고 연속 공백을 한 칸으로 줄인 뒤 100자에서 자른다.

그 값으로 getCachedPosts(pageNumber, sortOrder, keyword)를 기다린다. 이 함수는 "use cache", cacheTag("posts"), cacheLife("max")다. 인자가 캐시 키에 들어가므로 1페이지와 2페이지는 서로 다른 엔트리다. 미스이면 postService.listPosts가 Prisma로 읽는다.

listPosts의 페이지 크기는 20이다. prisma.post.count로 전체 개수를 세고, skip: (currentPage - 1) * 20, take: 20으로 findMany한다. 정렬은 createdAt이며 ASC만 오름차순이다. 키워드가 있으면 authorName 또는 body에 대소문자 무시 contains다. 반환의 hasNextPage는 currentPage < totalPages이고, hasPreviousPage는 currentPage > 1이다.

서버 페이지는 이 한 페이지 객체를 PostInfinityWrapper에 넘긴다. 같은 파일의 PostPaginataionWrapper 호출은 JSX 주석 안이라 실행되지 않는다. 번호 버튼 목록은 이 화면의 현재 경로가 아니다.

PostInfinityWrapper는 서버 컴포넌트이고, "use client"인 PostInfinityScrollList만 브라우저 상태가 된다. 스크롤이 다음 페이지를 부르면 router.replace가 주소만 바꾼다. 서버 컴포넌트가 그 주소의 한 페이지를 다시 그리고, 클라이언트 상태가 그 한 페이지를 기존 묶음에 붙인다. 브라우저가 1페이지부터 N페이지를 한 번의 fetch로 받지 않는다.

참고: Next.js useRouter, MDN URLSearchParams.

3. 이미 받은 페이지를 붙이는 계산

lib/postPageWindow.ts의 함수는 모두 동기다. createPostPageWindow는 서버가 넘긴 한 페이지로 창을 만든다.

queryKey는 `${sortOrder}:${keyword}`다. loadedKey는 `${pageNumber}:` 뒤에 그 페이지 글 id를 줄바꿈으로 이은 문자열이다.

PostInfinityScrollList는 useState의 초기값으로 창을 만든다. 그 다음 렌더마다 reducePostPageWindow를 호출하고, 반환값이 지금 상태와 다른 객체이면 렌더 중에 setWindowState를 호출한다. 이건 effect로 props를 복사하는 경로가 아니다. 서버에서 새 posts가 오면 그 렌더에서 창을 맞춘다.

reducePostPageWindow의 분기는 이 순서다.

1. queryKey와 loadedKey가 둘 다 같으면 지금 창을 그대로 반환한다. 같은 페이지 번호와 같은 id 목록이면 본문이나 좋아요 수가 달라도 창 안의 기존 객체를 유지한다.
2. queryKey가 다르면 한 페이지 창으로 바꾼다. 정렬이나 검색어가 바뀌면 이전에 붙여 둔 페이지는 사라진다. 정렬 링크는 postsListHref(1, "DESC" | "ASC", keyword)라 1페이지로 간다.
3. 같은 검색인데 그 페이지 번호가 이미 창에 있으면 그 그룹의 posts만 갈아끼운다. 갈아끼운 페이지가 맨 앞이면 hasPreviousPage, 맨 뒤이면 hasNextPage만 새 응답 값으로 바꾼다.
4. 페이지 번호가 맨 뒤 번호 + 1이면 그룹 배열 뒤에 붙이고 hasNextPage만 새 값으로 둔다. hasPreviousPage는 유지된다.
5. 페이지 번호가 맨 앞 번호 - 1이면 그룹 배열 앞에 붙이고 hasPreviousPage만 새 값으로 둔다. hasNextPage는 유지된다.
6. 그 외의 번호는 점프로 보고 한 페이지 창으로 바꾼다.

화면은 `view.groups`를 `data-post-page={group.page}`인 `div`로 그린다. 각 글은 `PostCard`다. 맨 앞 페이지에 이전 페이지가 있으면 높이 40px인 `이전 글을 불러오는 중...` 요소에 `topRef`를 단다. 맨 뒤에 다음 페이지가 있으면 높이 40px이고 배경이 `#ef4444`인 `데이터 로딩중...` 요소에 `bottomRef`를 단다. 이 빨간 배경은 소스의 `bg-red-500`이다.

참고: React, effect 없이 상태 맞추기, React useState.

4. 아래와 위를 보는 관찰

`useAdjacentPostPage`는 두 개의 effect를 쓴다. 둘 다 `useEffect`라 브라우저가 화면을 그린 뒤에 등록된다. 서버 렌더에서는 실행되지 않는다.

첫 effect는 `IntersectionObserver`를 만든다. 콜백은 동기 함수다. `loadingRef.current`가 참이면 바로 반환한다. 교차한 대상이 위 요소이면, `armPreviousRef`가 참이고 `hasPreviousPage`이며 `firstPage`가 1보다 클 때만 이전 주소로 간다. 아래 요소이고 `hasNextPage`이면 다음 주소로 간다. 이동은 `router.replace(postsListHref(...), { scroll: false })`다. `scroll: false`는 주소가 바뀌어도 스크롤을 맨 위로 올리지 말라는 Next.js 옵션이다. `postsListHref`는 `page`와 `sort`를 항상 넣고, 키워드가 있을 때만 `q`를 넣는다. 예는 `/posts?page=2&sort=DESC`다.

관찰은 조건이 있을 때만 한다. 위 요소는 `hasPreviousPage`일 때, 아래 요소는 `hasNextPage`일 때 `observe`한다. effect가 다시 실행되거나 컴포넌트가 없어지면 `disconnect`한다. effect 본문 첫 줄이 `loadingRef.current = false`이므로, 페이지 번호나 다음/이전 여부가 바뀌어 effect가 다시 돌면 잠금이 풀린다.

`IntersectionObserver`는 관찰을 시작하는 순간, 이미 보이는 요소에 대해서도 콜백을 한 번 부른다. 첫 화면에서 빨간 아래 요소가 뷰포트 안에 있으면 다음 페이지 주소로 바로 `replace`한다. 위 요소는 이 첫 콜백만으로는 이동하지 않는다. `armPreviousRef`의 처음 값이 `false`이기 때문이다.

둘째 effect가 그 깃발을 올린다. `scroll`에서 `window.scrollY`가 이전보다 작거나, `wheel`의 `deltaY`가 0보다 작으면 `requestPrevious`를 부른다. 두 리스너 모두 `{ passive: true }`라 `preventDefault`로 스크롤을 막지 않는다. `requestPrevious`는 깃발을 참으로 둔 다음, 위 요소의 `getBoundingClientRect`가 뷰포트와 겹칠 때만 곧바로 이전 페이지로 간다. 겹치지 않으면 깃발만 남고, 이후 첫 effect의 관찰 콜백이 위 요소의 교차를 처리할 수 있다. 그래서 목록을 처음 열자마자 위 센티널이 보인다는 이유만으로 0페이지를 요청하지 않는다.

참고: MDN IntersectionObserver, MDN getBoundingClientRect, MDN addEventListener의 passive, React useEffect.

5. 앞에 붙일 때 화면을 밀고, 현재 페이지 숫자를 보여 주는 일

이전 페이지 글은 DOM에서 지금 글보다 위에 들어간다. 높이가 위로 자라면 보이던 글이 아래로 밀린다. `usePreservePrependScroll`은 `useLayoutEffect`에서 이를 맞춘다. 그리기 전에 `document.documentElement.scrollHeight`를 읽고, `firstPage`가 이전보다 작으며 예전 맨 앞 페이지 번호가 `loadedPages`에 아직 있으면 `window.scrollBy(0, 새높이 - 저장해 둔 높이)`를 호출한다. 그 다음 지금 높이를 ref에 저장한다. `useEffect`로 하면 사용자가 한 프레임 밀린 위치를 본다. `useLayoutEffect`는 그 프레임 전에 스크롤을 고친다.

현재 페이지 배지는 `useVisiblePostPage`다. 화면 아래 가운데에 `N페이지 / 총페이지`를 고정한다. `data-post-page` 요소를 모두 읽고, 요소의 위쪽 좌표가 `window.innerHeight * 0.35` 이하인 것 중 마지막으로 읽힌 번호를 현재 페이지로 둔다. 스크롤 리스너도 `{ passive: true }`다. `anchorPage`가 바뀌면 effect가 아니라 렌더 중에 `visiblePage`를 그 번호로 맞춘다. `anchorPage`는 창을 새로 만들 때의 페이지 번호이고, 앞이나 뒤에 붙여도 `reducePostPageWindow`가 `anchorPage`를 다시 쓰지 않으므로 배지의 기준 리셋은 검색이 바뀌어 창이 한 페이지로 줄어들 때 일어난다.

참고: React useLayoutEffect, MDN scrollBy, MDN scrollHeight.

6. Spring으로 바뀐 범위와 아직 Prisma인 범위

어댑터를 고르는 함수는 `actions/authActions.ts`의 `authAdapter`다. `process.env.AUTH_ADAPTER === "prisma"`이면 `prismaAuthAdapter`, 아니면 `springAuthAdapter`다. 환경 변수가 없으면 Spring 쪽이다. 이 함수를 호출부가 고르지 않고 액션이 고른다.

호출	AUTH_ADAPTER=prisma	그 외
`loginWithCredentials`	`prismaAuthAdapter.login` → `userService.login`. Prisma `user.findUnique`	`springAuthAdapter.login` → `springAuthLogin` → `springFetch.post`
`linkAuthSocialAccount`	Prisma `userService.linkSocialAccount`	`springAuthAdapter.linkSocialAccount`가 다시 `prismaAuthAdapter.linkSocialAccount`를 부른다. 소셜 연결은 Spring 함수가 없다
`getCachedPosts` / `listPosts`	Prisma	Prisma. `matchApiUrl.post`를 읽지 않는다

`config/apiuri.ts`의 `matchApiUrl`은 객체 리터럴이다.

- `auth.login`: `/api/v1/auth/login`
- `auth.signup`: `/api/v1/auth/register`
- `auth.getUserData`: `/api/v1/auth/me`

- post.list, post.detail, post.create, post.update, post.delete: /api/v1/post/...

실제로 springFetch에 넘어가는 경로는 auth.login과 auth.getUserData뿐이다. auth.signup과 post 아래 다섯 문자열은 선언만 있다. springUserGetData는 export async function이지만 다른 파일이 이 함수를 import하지 않는다. 그래서 /api/v1/auth/me를 부르는 실행 경로는 아직 없다.

service/spring/sprignAuthService.ts 위쪽의 fetch(예시는 블록 주석 안이다. 실행되는 로그인은 그 아래의 springFetch.post다. service/postService.ts의 listPosts 주석에 있는 https://api.example.com/posts도 실행되지 않는다. 그 함수의 실행 경로는 prisma.post.count와 prisma.post.findMany다.

userService.ts의 주석 처리된 login은 springAccessToken이라는 이름을 적어 두었을 뿐 호출되지 않는다. 지금 실행되는 userService.login은 Prisma 사용자를 찾고 { ok: true, data }를 반환하며 Spring 토큰 필드가 없다. Prisma 어댑터는 그 결과에 빈 문자열 accessToken, refreshToken과 EXPIRES_IN_SECONDS(2시간)를 채운다.

참고: Next.js 서버 함수, MDN fetch.

7. fetchClient

lib/fetchClients.ts는 브라우저용 스크립트 파일이 아니다. 서버 모듈 sprignAuthService.ts가 springFetch를 import한다. createFetchClient는 팩토리 함수이고, 파일 맨 아래 export const springFetch = createFetchClient()가 모듈 평가 때 한 번 실행되어 메서드 다섯 개를 가진 객체를 만든다. 기본 baseUrl은 process.env.NEXT_PUBLIC_API_URL ?? ""이다. 이 기본값은 호출할 때마다 다시 읽지 않고, springFetch가 만들어질 때 한 번 고정된다. 다른 base URL로 만든 두 번째 클라이언트는 이 저장소에 없다.

반환 타입은 SpringApiResponseType<T | null>이다. types/response/springApiResponseType.ts의 필드는 success: boolean, code: number, message: string, data: T다. 클라이언트는 실패해도 예외를 던지지 않고 같은 모양으로 { success: false, code, message, data: null }을 반환한다. 호출부는 try/catch로 네트워크 실패를 구분하지 않고 response.success를 본다.

7.1 메서드

다섯 메서드는 모두 안쪽 request로 간다.

메서드	HTTP	본문 인자
get(path, options?)	GET	넘기지 않는다. request의 body는 undefined
post(path, body?, options?)	POST	두 번째 인자
put(path, body?, options?)	PUT	두 번째 인자
patch(path, body?, options?)	PATCH	두 번째 인자
delete(path, body?, options?)	DELETE	두 번째 인자. GET이 아니므로 body가 있으면 JSON이나 FormData로 실릴 수 있다

options의 타입 SpringRequest는 params, headers, signal, timeout만 있다. credentials, cache, redirect, next 옵션은 이 타입에 없다. fetch에는 method, headers, body, signal만 넘기므로 나머지 Request 기본값이 적용된다. Fetch 표준의 기본 redirect는 follow, 기본 credentials는 same-origin이다.

7.2 주소

buildUrl은 세 조각을 합친다.

1. path가 http:// 또는 https://로 시작하면 base URL을 붙이지 않는다. 판별은 /^https?:\/\//i라 대문자 HTTPS://도 절대 주소다.
2. 상대 경로이면 base 끝의 / 하나를 지우고, path 앞에 /가 없으면 붙인다. https://api.example.com/와 api/v1/auth/login은 https://api.example.com/api/v1/auth/login이 된다.
3. params가 있으면 URLSearchParams로 쿼리를 붙인다. 값이 null이거나 undefined인 키는 뺀다. 그 외는 String(value)다. 숫자 0은 "0", 불리언 false는 "false"다. 인코딩은 URLSearchParams가 한다.

base URL이 빈 문자열이고 path가 절대 주소도 아니면 fetch를 호출하지 않는다. 반환은 code: 0, message: "API 주소가 설정되지 않았습니다.", data: null이다.

7.3 헤더와 본문

시작 헤더는 Accept: application/json이다. options.headers가 그 위에 덮인다. Headers 객체, [key, value][], 일반 객체를 모두 평평한 레코드로 바꾼 뒤 전개한다. 호출자가 Accept를 넘기면 기본값을 바꾼다.

body가 undefined이거나 메서드가 GET이면 init.body를 넣지 않는다. FormData이면 그대로 init.body에 넣고 Content-Type을 코드가 추가하지 않는다. 브라우저와 undici가 multipart/form-data와 boundary를 넣게 두기 위해서다. 호출자가 직접 Content-Type을 넘겼으면 그 값은 지워지지 않

고 남는다. FormData가 아닌 본문은 JSON.stringify하고, 레코드에 Content-Type과 content-type이 둘 다 없을 때만 Content-Type: application/json을 넣는다.

springAuthLogin은 두 번째 인자로 { email: email.trim(), password }를 넘긴다. 이건 FormData가 아니므로 JSON 문자열과 Content-Type: application/json이 붙는다. springUserGetData는 GET이라 본문이 없고, headers.Authorization만 Bearer ${accessToken}으로 넘긴다.

7.4 중단과 시간 제한

매 요청마다 new AbortController()를 만든다. fetch에 넘기는 signal은 호출자가 준 options.signal이 아니라 이 컨트롤러의 signal이다.

timeout이 0보다 큰 수일 때만 setTimeout이 그 밀리초 뒤에 controller.abort()를 호출한다. springAuthLogin과 springUserGetData는 timeout을 넘기지 않으므로 이 타이머는 생기지 않는다. options.signal이 있으면 그 signal의 abort 이벤트에 { once: true }로 리스너를 달아 같은 컨트롤러를 중단한다. 리스너를 달기 전에 이미 aborted인 signal은 이 코드가 signal.aborted를 보지 않으므로, 새 abort 이벤트가 없으면 컨트롤러가 중단되지 않는다.

fetch가 끝나거나 예외가 나면 finally에서 타이머가 있을 때만 clearTimeout한다. 중단과 네트워크 실패는 같은 catch로 모인다. AbortError와 연결 실패를 구분하지 않고 code: 0, message: "일시적인 오류가 발생했습니다. 잠시 후 다시 시도해 주세요."를 반환한다.

7.5 응답을 Spring 모양으로 읽는 순서

readBody는 Content-Type 헤더를 본다.

- application/json을 포함하면 response.json()을 기다린다. 파싱이 거절되면 undefined다.
- text/로 시작하면 response.text()다.
- 그 외는 undefined다. 본문을 blob으로 읽지 않는다.

그 다음 toSpringBody가 객체이고 success가 불리언일 때만 Spring 응답으로 인정한다. code가 숫자면 그대로 쓰고, 아니면 HTTP 상태 코드를 쓴다. message가 문자열이면 그대로 쓰고, 아니면 ""다. data가 없으면 null이다. success: false인 JSON도 이 단계에서 그대로 반환한다. HTTP 상태가 4xx여도 Spring 모양이면 "요청에 실패했습니다."로 바꾸지 않는다.

Spring 모양이 아니고 response.ok가 거짓이면 code는 HTTP 상태, message는 "요청에 실패했습니다."다. Spring 모양이 아니고 response.ok가 참이면 success: true, code는 HTTP 상태, message는 "", data는 파싱 결과 또는 null이다. 텍스트 본문이면서 2xx이면 data는 그 문자열이다.

이 순서가 로그인 판별과 만난다. springAuthLogin은 response.success가 거짓이거나 isLoginData가 거짓이면 { ok: false, message }를 반환한다. isLoginData는 accessToken이 있고 userId가 숫자이며 email이 있을 때만 참이다. 메시지는 response.message가 비어 있으면 "이메일 또는 비밀번호가 올바르지 않습니다."다. 성공이면 { ok: true, data: response.data }다.

springUserGetData는 토큰 문자열이 비어 있으면 fetch를 호출하지 않고 { ok: false, message: "인증 정보를 찾을 수 없습니다." }를 반환한다. 호출한 경우에는 같은 isLoginData를 쓰고, 실패 문구의 기본값은 "사용자 정보를 조회할 수 없습니다."다.

참고: MDN fetch, MDN Request, MDN Headers, MDN FormData, MDN AbortController, MDN URLSearchParams, MDN Response.json, MDN setTimeout.

8. 로그인 응답이 세션에 남기는 토큰

Spring 로그인 데이터의 타입 SpringLoginData는 accessToken, refreshToken, expiresIn, tokenType, userId, email, name, role, platformRole, avatarUrl, presenceStatus, statusMessage, accountStatus, companyId를 적는다. userId와 companyId는 숫자다. avatarUrl, statusMessage, companyId는 null일 수 있다.

springAuthAdapter.login은 이를 AuthAccount로 바꾼다. id는 String(data.userId), profileImageUrl과 avatarUrl은 null이면 "", companyId는 숫자면 toString()이고 null이면 ""다. accessToken 자리에는 Spring이 준 액세스 토큰이 들어간다.

그 다음은 진입점마다 다르다.

NextAuth 자격 증명 authorize는 loginWithCredentials의 user를 그대로 User로 반환한다. jwt 콜백은 account.provider === "credentials"일 때 이렇게 나눈다.

- token.springAccessToken = user.accessToken. Spring 액세스 토큰이다.
- token.refreshToken = user.refreshToken.
- token.accessToken = await signAccessToken(...). 이건 Spring 토큰이 아니다. lib/accessJwt.ts가 NEXTAUTH_SECRET으로 HS256 JWT를 만들고, sub는 사용자 id, 만료는 60 * 60 * 2초, 클레임은 email과 name이다.
- token.expiresIn = user.expiresIn. Spring이 준 초다. 로컬 JWT의 2시간과 별도 필드다.

session 콜백은 session.accessToken과 session.user.accessToken에 token.accessToken을 넣는다. 즉 세션에 노출되는 액세스 토큰은 로컬 HS256 JWT다. token.springAccessToken을 session에 복사하는 줄은 없다. types/next-auth.d.ts의 JWT 인터페이스에는 springAccessToken?이 있고, 그 필드를 읽어 가는 다른 실행 코드는 없다.

`POST /api/auth/login`도 `loginWithCredentials`를 부른 뒤 `signAccessToken`으로 새 JWT를 만들어 JSON `data.accessToken`에 넣는다. `data.expiresIn`은 `result.user.expiresIn`이라 Spring 어댑터일 때는 Spring의 `expiresIn`이다. 이 JSON에는 Spring `accessToken`과 `refreshToken` 필드가 없다. 잘못된 본문은 400, 로그인 실패는 401이며 `Access-Control-Allow-Origin`은 `*`다. `OPTIONS`는 204다.

Google과 Kakao는 `jwt` 콜백의 소셜 분기에서 `linkAuthSocialAccount`를 부른다. Spring 어댑터여도 이 함수는 Prisma 연결로 내려간다. 소셜 분기에서는 `signAccessToken` 결과만 `token.accessToken`에 두고 `token.springAccessToken`을 넣지 않는다.

참고: MDN Fetch CORS, ECMA-262 JSON.stringify.

9. 이 코드가 하지 않는 일

- 무한 스크롤이 `springFetch.get(matchApiUrl.post.list)`를 호출하지 않는다. 글 목록은 `getCachedPosts`와 Prisma다.
- 무한 스크롤이 `useState`로 글 배열을 이어 붙이며 브라우저에서 page 쿼리를 누적 `fetch`하지 않는다. 누적은 `reducePostPageWindow`이고, 다음 데이터는 주소의 `page`가 바뀐 서버 렌더가 한 페이지씩 준다.
- `PostPaginataionWrapper`는 `posts/page.tsx`에서 주석이라 함께 그려지지 않는다.
- `matchApiUrl.auth.signup`과 `matchApiUrl.post`의 다섯 경로는 `springFetch`에 전달되지 않는다.
- `springUserGetData`는 정의만 있고 호출부가 없다.
- `fetchClient`는 실패를 throw하지 않고, 시간 초과와 연결 실패를 서로 다른 `code`로 나누지 않는다.
- 세션 객체에 Spring 액세스 토큰을 실어 클라이언트 컴포넌트로 넘기는 코드는 없다.